

Srija Yadav Vuppula

+1 (706) 254-7688 | vuppulasrija16@gmail.com | LinkedIn | GitHub

Google Summer of Code 2026 Proposal

Organization: 52°North

Project: Weather Routing Tool – Improve Test Framework

ABOUT ME

The University of Georgia

Master of Science in Computer Science

Athens, GA

Jan 2025 – Dec 2026

- I am a graduate student in Computer Science at the University of Georgia. The kind of software work I enjoy most is the kind that improves a codebase in a lasting way: debugging real behavior, tightening tests, reducing ambiguity, and making future contributions easier for everyone who works on the project after me.
- That is exactly why the Weather Routing Tool test-framework project stood out to me. It is practical, specific, and directly useful. The work is not about adding complexity for its own sake; it is about improving reliability, structure, and contributor experience in a project that already does meaningful technical work.
- I wanted to start this proposal from the terminal, not just the ideas page. After cloning the repository and fixing a few local environment quirks, I traced the routing logic and got the relevant tests running. From there, I started pushing focused contributions. That process gave me a much more realistic understanding of the current test framework and the improvements that would matter most.
- Overall, I believe I am a strong fit for this project because I like careful engineering work. I enjoy contributions that improve code quality, reduce confusion, and leave the repository in a better state than I found it.

MOTIVATION FOR THIS PROJECT

- Software quality and maintainability matter most to me in repositories where people need to trust technical behavior, not just add features. The Weather Routing Tool fits that well. It is a real engineering codebase with routing logic, scientific constraints, and testing needs that directly affect how safely the project can evolve.
- This specific test-framework project focuses on a concrete bottleneck. Better test categorization, stronger regression coverage, and clearer contributor workflows would make the repository easier to extend and easier to trust. That kind of work is immediately useful, which is a big part of why I was drawn to it.
- What I want to contribute is straightforward: a WRT test suite that is easier to run, easier to understand, and more dependable during development. I am not only interested in adding tests. I want to improve the structure around the suite so changes to routing code can be validated with more confidence and less friction.

PROJECT DESCRIPTION

- **Project:** Weather Routing Tool – Improve Test Framework
- **Ideas Page:** 52°North GSoC 2026 Project Ideas
- **Mentors:** Martin Pontius and Katharina Demmich
- **Problem Statement:** The Weather Routing Tool already has a meaningful pytest-based test suite, but there is still room to improve structure, readability, separation of responsibilities, and developer workflow. In particular, the suite would benefit from a cleaner split between unit and integration tests, clearer handling of monitoring or plot-producing tests, stronger regression checks for reproducibility, and more contributor-friendly test commands and documentation.
- **Project Goal:** The goal of this project is to improve the WRT test framework so that it becomes more maintainable and more useful in practice. That includes organizing tests more clearly, isolating expensive or monitoring-style runs from the default workflow, improving correctness where current tests are weak or fragile, and adding reproducible route-level regression checks that support future development.

- **Why This Matters:** For a routing tool, weak or confusing tests do more than slow down CI. They also make code review harder, make regressions harder to interpret, and raise the cost of changing routing behavior safely. Improving the test framework therefore improves both code quality and contributor productivity.
- **Project Scope:** I see this as a focused 175-hour engineering project. The work is not about a large redesign. It is about improving the current framework in a technically careful, incremental, and reviewable way so the repository becomes easier to maintain and easier to trust.

CODING CHALLENGE AND CURRENT CONTRIBUTIONS

- **Starting Point:** Since 52°North expects applicants to begin with the coding challenge, I used the repository itself as the starting point for this proposal. The local setup came first: Python 3.11, dependency installation, and a working test environment. That early setup work was useful because it exposed the actual friction points in the project instead of leaving me with only a high-level idea of the test framework.
- **Repository Familiarization:** Work on the challenge quickly led into the parts of the codebase most relevant to this project. The routing logic in `WeatherRoutingTool/algorithms/genetic/problem.py` and the corresponding tests in `tests/test_genetic.py` became the main focus. Looking through those files made the current strengths and gaps of the suite much easier to see: there is already a solid base, but the structure can be clearer, some tests need stronger organization, and the path toward reproducible regression checks needs to be more deliberate.
- **Coding-Challenge Work Completed:** The required `RoutingProblem.get_power` unit-test task is already done and integrated into the WRT pytest suite. Beyond that, I also completed a local WRT run using the recommended configuration direction for the challenge, collected execution evidence, and prepared the proposal materials around the actual repository artifacts rather than describing hypothetical work.
- **Weather Data Used:** The weather dataset used for the local run is `tests/data/tests_weather_data.nc`. It is referenced by the WRT test configuration files, exists in the repository setup I used locally, and contains time-, latitude-, and longitude-based weather/ocean variables suitable for a WRT test run.
- **Configuration Used:** The base local configuration file for the coding challenge is `tests/config.tests.json`. It defines the route and input data, including `DEFAULT_ROUTE`, `WEATHER_DATA`, `ROUTE_PATH`, and `BOAT_TYPE = "direct_power_method"`. In the local test setup, `ALGORITHM_TYPE` is set to `genetic` in `tests/test_config.py`, which matches the coding-challenge recommendation.
- **Execution Evidence:** WRT was executed locally from the CLI using `tests/config.gsocc_run.json`. The run initialized the weather module from `tests/data/tests_weather_data.nc`, used the direct power method for ship modeling, and started the genetic routing procedure. The terminal log is included in the appendix as part of the code challenge report.
- **Required Unit Test:** The `RoutingProblem.get_power` logic now has a dedicated unit test in `tests/test_genetic.py`. That test uses a controlled dummy boat and mocked waypoint data to verify the computed fuel sum and the returned fields. I included the code in the appendix because it is the most direct part of my coding-challenge solution.
- **Verification:** Local verification was done with `./venv/bin/pytest tests/test_genetic.py -q`. The result was 11 passed, 1 skipped. This includes the new `get_power` test added on branch `test/get-power-unit-test` at commit `e2a310c`.
- **Additional Contributions:** The coding challenge also led naturally into three non-duplicate follow-up contributions that fit the broader direction of this project:
 - PR #190: added pytest unit/integration markers and test-running documentation.
 - PR #191: fixed weather-test collection and assertions for `get_u` and `get_v`.
 - PR #192: marked plot-producing genetic tests as manual so they do not run in the normal path by default.

Those changes matter because they are not just isolated patches; they point directly toward the kind of test-framework cleanup this project is about.

TECHNICAL APPROACH

- **Overall Strategy:** This project breaks into four main parts: auditing the current suite, cleaning up the test structure, adding reproducible regression coverage, and finishing with documentation plus workflow polish. That keeps the work incremental and reviewable instead of turning a medium-sized project into a vague cleanup effort.
- **Stage 1: Audit and Categorization:** A clear map of the existing suite comes first. I want to identify which tests are fast unit tests, which behave more like integration tests, which ones produce figures or monitoring artifacts, and which parts of the repository still have weak or fragile coverage. Assertion quality, repeated setup, naming consistency, and runtime cost all belong in this audit. Without that baseline, it is too easy to improve the wrong things first.
- **Stage 2: Structural Cleanup:** This involves deliberate pytest marker usage, cleaner separation between default and optional runs, better grouping of related tests, and stripping out repeated setup where fixtures genuinely improve readability. Monitoring-style tests should stay easy to run on demand, but they should not make the normal development path heavier than it needs to be.
- **Stage 3: Regression Coverage:** Route-level regression testing is the next priority. Reproducibility matters most here. I want to use a small number of carefully chosen configurations that reflect real WRT behavior, stay practical in runtime, and fail in ways that are actually useful during development.
- **Stage 4: Documentation and Cleanup:** Contributor-facing commands, explicit guidance on which tests run by default versus on demand, and final cleanup all belong here. The goal is to leave the improved framework easy to use and easy to understand after the project ends.
- **Implementation Style:** I would rather deliver a smaller set of strong, well-integrated improvements than stretch the project into a broad rewrite. For a 175-hour project, the best result is something maintainers can trust, review comfortably, and keep building on after GSoC.

DETAILED IMPLEMENTATION PLAN

- **Work Package 1 – Test Framework Audit:** I will begin by reviewing the existing test suite to classify current tests by purpose, runtime cost, and quality. This phase will produce a concrete map of what is already covered, where boundaries between test categories are unclear, and which parts of the suite are currently awkward to run or maintain.
- **Work Package 2 – Unit / Integration / Manual Separation:** The next phase will focus on strengthening the separation of fast default tests from slower integration and monitoring-style tests. Some of this work has already begun in my pre-proposal contributions, and during GSoC I would extend that into a more consistent framework across the relevant parts of the repository.
- **Work Package 3 – Regression Scenarios for Reproducibility:** After the suite structure is more stable, I will design a small set of reproducible route-level regression scenarios. The emphasis will be on keeping them representative enough to be useful while still being affordable in runtime. I want failures in these tests to be interpretable and developer-friendly rather than noisy.
- **Work Package 4 – Cleanup and Contributor-Facing Improvements:** The last major phase will focus on documentation, test commands, readability improvements, and final cleanup. This includes making it easier for a new contributor to understand which tests should run by default, which tests are optional, and how to validate changes without guessing.

EXPECTED DELIVERABLES

- **Deliverable 1:** A clearer and more maintainable pytest-based test structure for the Weather Routing Tool.
- **Deliverable 2:** A cleaner separation between unit tests, integration tests, and monitoring-style or manual tests.
- **Deliverable 3:** Additional regression coverage for selected reproducible routing scenarios, designed with practical runtime in mind.
- **Deliverable 4:** Targeted correctness improvements in weak or fragile tests, including better assertions and clearer setup.

- **Deliverable 5:** Updated contributor-facing documentation for running test categories and understanding the expected workflow.

WEEKLY TIMELINE

- **Community Bonding Period:** This period is for scope alignment, discussion with mentors, and turning the proposal into a concrete sequence of implementation tasks. I also want the local workflow to be fully stable before coding begins so the project starts with real momentum rather than setup delays.
- **Week 1:** Kicking off with a structured audit of the current test framework: test categories, repeated setup patterns, weak assertions, and the main pain points in the default workflow.
- **Week 2:** The audit results should narrow into a clear priority order here. This is also the right point to confirm with mentors which structural improvements should land first and which ones are better treated as stretch work.
- **Week 3:** Structural cleanup takes over. Marker usage, grouping of related tests, naming consistency, and trimming duplicated setup all belong here.
- **Week 4:** More cleanup, but now with the first round already validated. By this point I want the suite to feel more deliberate and less patchy than it does now.
- **Week 5:** Separating monitoring-style or plot-producing tests from the normal development path. They should still be easy to run, just not part of the default workflow.
- **Week 6 – Midterm:** By midterm, the target is a noticeably cleaner and more usable test structure, with clearer boundaries between test categories and a contributor workflow that already feels lighter.
- **Week 7:** Regression work starts here. The focus is on choosing a small number of reproducible route scenarios that are representative enough to be useful without becoming too expensive.
- **Week 8:** Expanding that regression layer and improving the way failures are interpreted. If a regression test breaks, the output should help a contributor understand what changed rather than simply reporting that something is wrong.
- **Week 9:** Review integration and cleanup. This includes tightening assertions, adjusting tests based on mentor feedback, and making sure the structural and regression work still fit together cleanly.
- **Week 10:** Writing out the contributor-facing documentation so the optional test paths, normal workflows, and expected commands are obvious to newcomers.
- **Week 11 – Code Freeze:** No ambitious new scope here. This time is for stabilization, small fixes, final review feedback, and making the finished work solid.
- **Week 12:** Final validation, cleanup, packaging the work clearly, and documenting natural follow-up tasks that would make sense after the project ends.

POTENTIAL CHALLENGES AND MITIGATION

- **Balancing Coverage and Runtime:** Regression coverage is useful only if it stays practical. I plan to keep route-level scenarios small, representative, and controlled so they add confidence without making the suite too expensive to run.
- **Scope Control:** Test-framework work can expand quickly in many directions at once. To avoid that, I will prioritize the suite structure and category separation first, then regression coverage, and then any optional workflow improvements.
- **Cross-Cutting Repository Effects:** Even small test-structure changes can affect the broader contributor workflow. I plan to work in small, reviewable pieces so that each improvement can be validated before the next one builds on top of it.
- **Environment and Dependency Issues:** Scientific Python repositories often bring setup and runtime friction. Since I have already recreated the environment and worked through local execution issues, I have a stronger starting point, but I still want the project plan to remain robust if setup-related problems reappear.

AVAILABILITY AND COMMUNICATION

- **Availability:** I expect to spend around 15–18 hours per week on this project during the coding period, which aligns well with the proposed 175-hour scope.
- **Known Conflicts:** At the time of writing, I do not expect any major long-duration conflicts that would interfere with steady progress during the coding period.
- **Communication Style:** I am comfortable working with regular updates, review-driven iteration, and small technical questions as they arise. For a project like this, I think early alignment and smaller reviewable steps are much more valuable than large batches of changes delivered all at once.

AFTER GSoC

- **Continued Involvement:** I do not see this as a one-time summer patch set. If the collaboration goes well, I would like to continue contributing to the Weather Routing Tool after GSoC, especially in areas related to testing, contributor workflow, and incremental maintainability improvements.
- **Long-Term Interest:** This project is a strong entry point for me into the 52°North community because it matches the kind of engineering work I genuinely enjoy. I would like to continue building on that beyond the official GSoC period.

REFERENCES

- 52°North, *Applying for GSoC @ 52°North*. <https://52north.org/outreach-dissemination/google-summer-of-code/applying-for-gsoc-52north/>
- 52°North, *Project Ideas for Google Summer of Code*. <https://52north.org/outreach-dissemination/google-summer-of-code/project-ideas/>
- 52North, *WeatherRoutingTool Repository*. <https://github.com/52North/WeatherRoutingTool>
- 52North, *WeatherRoutingTool Documentation*. <https://52north.github.io/WeatherRoutingTool/>
- 52North, *WeatherRoutingTool Documentation – Configuration*. <https://52north.github.io/WeatherRoutingTool/source/configuration.html>
- 52North, *WeatherRoutingTool Documentation – Tests*. <https://52north.github.io/WeatherRoutingTool/source/guidelines/tests.html>
- 52North, *WeatherRoutingTool Documentation – Genetic Algorithm*. <https://52north.github.io/WeatherRoutingTool/source/algorithms/genetic.html>
- 52North, *WeatherRoutingTool Documentation – Consumption Models / Direct Power Method*. <https://52north.github.io/WeatherRoutingTool/source/consumption-models/index.html>

APPENDIX A – CODING CHALLENGE EVIDENCE

- **A.1 Weather Data and Config Evidence:** The screenshots below support the weather-data and configuration setup used for the coding challenge.
- **A.2 WRT Run Evidence:** The run log shows that WRT initialized the weather module from the local dataset, used the direct power method for ship modeling, and started the genetic routing procedure.
- **A.3 Test Evidence:** The final screenshot shows the passing pytest run for `tests/test_genetic.py`.
- **A.4 Required Code Snippet:** The required `RoutingProblem.get_power` unit test is included below exactly as implemented.

```
srijayadav@Srijas-MacBook-Air WeatherRoutingTool % ../venv/bin/python - <<'PY'
import xarray as xr
ds = xr.open_dataset("tests/data/tests_weather_data.nc")
print(ds)
PY
<xarray.Dataset> Size: 29MB
Dimensions:
    (time: 32, depth: 1, latitude: 61,
     longitude: 169,
     height_above_ground: 2)

Coordinates:
  * time          (time) datetime64[ns] 256B 2025-...
  * depth         (depth) float32 4B 0.494
  * latitude      (latitude) float64 488B 37.0 .....
  * longitude     (longitude) float64 1kB 1.0 ... ...
  * height_above_ground (height_above_ground) float32 8B ...
    LatLon_Projection int32 4B ...
Data variables:
    vttotal          (time, depth, latitude, longitude) float32 1MB ...
    utotal           (time, depth, latitude, longitude) float32 1MB ...
    thetao           (time, depth, latitude, longitude) float32 1MB ...
    so              (time, depth, latitude, longitude) float32 1MB ...
    VMDR            (time, latitude, longitude) float64 3MB ...
    VHM0            (time, latitude, longitude) float64 3MB ...
    VTPK            (time, latitude, longitude) float64 3MB ...
    Temperature_surface (time, latitude, longitude) float64 3MB ...
    u-component_of_wind_height_above_ground (time, height_above_ground, latitude, longitude) float64 5MB ...
    v-component_of_wind_height_above_ground (time, height_above_ground, latitude, longitude) float64 5MB ...
    Pressure_reduced_to_MSL_msl (time, latitude, longitude) float64 3MB ...
Attributes:
    institution:  MERCATOR OCEAN
    title:        hourly mean merged surface currents from oceanic circulatio...
    source:       MERCATOR GLO12, mfwamglo, FES2014
    producer:     CMEMS - Global Monitoring and Forecasting Centre
    Conventions:  CF-1.6
    references:   http://marine.copernicus.eu
    contact:      servicedesk.cmems@mercator-ocean.eu
    credit:       E.U. Copernicus Marine Service Information (CMEMS)
srijayadav@Srijas-MacBook-Air WeatherRoutingTool % █
```

Figure 1: Local weather dataset inspection for `tests/data/tests_weather_data.nc`.

```
[srijayadav@Srijas-MacBook-Air WeatherRoutingTool % sed -n '1,40p' tests/config.tests.json
{
  "DEFAULT_ROUTE": [38.192, 13.392, 41.349, 2.188],
  "DEPARTURE_TIME": "2025-04-01T11:11Z",
  "DEFAULT_MAP": [37.1, 1.1, 41.9, 14.9],
  "BOAT_DRAUGHT_AFT": 10,
  "BOAT_DRAUGHT_FORE": 10,
  "INTERMEDIATE_WAYPOINTS": [[43.171, -132.077], [55.483, -141.686]],
  "CONSTRAINTS_LIST": ["land_crossing_global_land_mask", "water_depth", "via_waypoints"],
  "TIME_FORECAST": 20,
  "ROUTING_STEPS": 60,
  "DELTA_TIME_FORECAST": 3,
  "DELTA_FUEL": 3000,

  "COURSES_FILE": "tests",
  "DEPTH_DATA": "tests/data/tests_depth_data.nc",
  "WEATHER_DATA": "tests/data/tests_weather_data.nc",
  "ROUTE_PATH": "tests",

  "DATA_MODE": "from_file",

  "ROUTER_HDGS_SEGMENTS": 4,
  "ROUTER_HDGS_INCREMENTS_DEG": 1,
  "ISOCHRONE_PRUNE_SECTOR_DEG_HALF": 91,
  "ISOCHRONE_PRUNE_SEGMENTS": 20,

  "ISOCHRONE_MINIMISATION_CRITERION": "squaredist_over_disttodest",
  "ISOCHRONE_NUMBER_OF_ROUTES": 1,

  "GENETIC_NUMBER_GENERATIONS": 20,
  "GENETIC_NUMBER_OFFSPRINGS": 2,
  "GENETIC_POPULATION_SIZE": 20,
  "GENETIC_POPULATION_TYPE": "grid_based",

  "BOAT_TYPE": "direct_power_method",
  "BOAT_BREADTH": 32,
  "BOAT_FUEL_RATE": 167,
  "BOAT_HBR": 30,
  "BOAT_LENGTH": 180,
  "BOAT_SMCR_POWER": 6502,
  "BOAT_SMCR_SPEED": 6,
srijayadav@Srijas-MacBook-Air WeatherRoutingTool %
```

Figure 2: Base local configuration file used for the coding challenge.

```
[srijayadav@Srijas-MacBook-Air WeatherRoutingTool % sed -n '140,150p' tests/test_config.py
def test_boat_speed_arrival_time_waypoint_optimisation_failure(boat_speed, arrival_time, mut_type, cross_type, ierr):
    config_data, _ = load_example_config()
    config_data["BOAT_SPEED"] = boat_speed
    config_data["ARRIVAL_TIME"] = arrival_time
    config_data["GENETIC_MUTATION_TYPE"] = mut_type
    config_data["GENETIC_CROSSOVER_TYPE"] = cross_type
    config_data["ALGORITHM_TYPE"] = "genetic"
    error_str_list = [
        "Please specify EITHER the boat speed OR the arrival time but not both.",
        "Please specify EITHER the boat speed OR the arrival time.",
    ]
srijayadav@Srijas-MacBook-Air WeatherRoutingTool %
```

Figure 3: Local test setup showing ALGORITHM_TYPE = genetic.

```

srijayadav@Srijas-MacBook-Air WeatherRoutingTool % ./venv/bin/python cli.py -f "$(pwd)/tests/config.gsocc_run.json"
2026-03-30 06:40:35,872 - WRT.Config : INFO arrival time: 2025-04-01 23:11:00
2026-03-30 06:40:35,872 - WRT.Config : INFO speed: 6.0
2026-03-30 06:40:35,872 - WRT.Config : INFO Algorithm run mode: speed optimisation for fixed route or simultaneous waypoint and speed optimisation.

-----
2026-03-30 06:40:35,872 - WRT.weather : INFO Initialising weather
2026-03-30 06:40:35,872 - WRT.weather : INFO Reading weather data from file: tests/data/tests_weather_data.nc
2026-03-30 06:40:35,872 - WRT.weather : INFO time resolution: 3:00:00 hours
2026-03-30 06:40:35,878 - WRT.weather : INFO forecast from 2025-04-01 09:00:00 to 2025-04-02 09:00:00
2026-03-30 06:40:35,878 - WRT.weather : INFO nof time steps 8
-----

2026-03-30 06:40:35,878 - WRT : INFO Initialising ship
2026-03-30 06:40:35,878 - WRT : INFO Use direct power method for modeling fuel consumption.
2026-03-30 06:40:35,879 - WRT.ship : INFO The boat speed provided is assumed to be the speed that corresponds to 75% SMCR power.
2026-03-30 06:40:35,879 - WRT.ship : INFO SMCR power: 4876.5 kW
2026-03-30 06:40:35,879 - WRT.ship : INFO SMCR speed: 6.0 m / s
2026-03-30 06:40:35,879 - WRT.ship : INFO fuel rate: 0.167 kg / (h kW)
-----

2026-03-30 06:40:35,879 - WRT.Constraints: INFO Downloading depth data from file: tests/data/tests_depth_data.nc
2026-03-30 06:40:35,881 - WRT.Constraints: INFO Downloading depth data from file: tests/data/tests_depth_data.nc
-----

2026-03-30 06:40:35,883 - WRT.Constraints: INFO Constraints activated:
2026-03-30 06:40:35,883 - WRT.Constraints: INFO no land crossing
2026-03-30 06:40:35,883 - WRT.Constraints: INFO minimum water depth=30.0m
2026-03-30 06:40:35,883 - WRT.Constraints: INFO intermediate waypoints activated for: (43.171, -132.077)
2026-03-30 06:40:35,883 - WRT.Constraints: INFO intermediate waypoints activated for: (55.483, -141.686)
-----

2026-03-30 06:40:35,883 - WRT : INFO Initialising and starting routing procedure. For log output check the files 'info.log' and 'performance.log'.
2026-03-30 06:40:35,895 - WRT.genetic : INFO Initializing Routing.....
2026-03-30 06:40:35,895 - WRT.genetic : INFO route from (38.192, 13.392) to (41.349, 2.188)

```

Figure 4: CLI run of WRT using tests/config.gsocc_run.json.

```

[srijayadav@Srijas-MacBook-Air WeatherRoutingTool % ./venv/bin/pytest tests/test_genetic.py -q
.....S..
----- warnings summary -----
.venv/lib/python3.11/site-packages/webob/compat.py:5
/Users/srijayadav/GSoC_2026/52North/WeatherRoutingTool/.venv/lib/python3.11/site-packages/webob/compat.py:5: DeprecationWarning: 'cgi' is deprecated and slated for removal in Python 3.13
from cgi import parse_header

tests/test_genetic.py::test_constraint_violation_repair
tests/test_genetic.py::test_constraint_violation_repair
/Users/srijayadav/GSoC_2026/52North/WeatherRoutingTool/.venv/lib/python3.11/site-packages/numpy/_core/fromnumeric.py:3824: RuntimeWarning: Mean of empty slice
return methods._mean(a, axis=axis, dtype=dtype,

tests/test_genetic.py::test_constraint_violation_repair
tests/test_genetic.py::test_constraint_violation_repair
/Users/srijayadav/GSoC_2026/52North/WeatherRoutingTool/.venv/lib/python3.11/site-packages/numpy/_core/_methods.py:142: RuntimeWarning: invalid value encountered in scalar divide
ret = ret.dtype.type(ret / rcount)

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
===== short test summary info =====
SKIPPED [1] tests/test_genetic.py:340: Test needs modified route array.
11 passed, 1 skipped, 8 warnings in 4.70s
srijayadav@Srijas-MacBook-Air WeatherRoutingTool %

```

Figure 5: Passing pytest run for tests/test_genetic.py.

APPENDIX B – ROUTINGPROBLEM.GET_POWER UNIT TEST

```

1 def test_routingproblem_get_power():
2     departure_time = datetime(2025, 4, 1, 12, 0, 0)
3     arrival_time = departure_time + timedelta(seconds=30)
4
5     class DummyShipParams:
6         def get_fuel_rate(self):
7             return np.array([2.0, 3.0]) * (u.kg / u.second)
8
9     class DummyBoat:
10         def get_ship_parameters(self, courses, lats, lons, time, speed):
11             return DummyShipParams()
12
13     problem = RoutingProblem(
14         departure_time=departure_time,
15         arrival_time=arrival_time,
16         boat=DummyBoat(),
17         boat_speed=10.0,
18         constraint_list=[],
19         objectives={"fuel_consumption": 1, "arrival_time": 1},
20     )
21
22     route = np.array([

```

```

23     [35.199, 15.490, 5.0],
24     [34.804, 16.759, 6.0],
25     [34.447, 18.381, 7.0],
26 ])
27
28 original = RouteParams.get_per_waypoint_coords
29
30 def fake_get_per_waypoint_coords(lons, lats, departure_time, bs):
31     return {
32         "courses": np.array([10.0, 20.0]),
33         "start_lats": np.array([35.199, 34.804]),
34         "start_lons": np.array([15.490, 16.759]),
35         "start_times": [
36             departure_time,
37             departure_time + timedelta(seconds=10),
38         ],
39         "travel_times": np.array([10.0, 20.0]) * u.second,
40     }
41
42 RouteParams.get_per_waypoint_coords = staticmethod(fake_get_per_waypoint_coords)
43 try:
44     result = problem.get_power(route)
45 finally:
46     RouteParams.get_per_waypoint_coords = original
47
48 assert np.isclose(result["fuel_sum"].value, 80.0)
49 assert result["time_obj"] == 1
50 assert "shipparams" in result

```